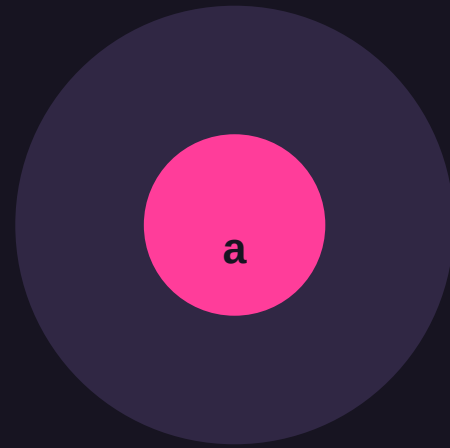




THE VENUE LOYALTY WALLET

AIKIA.WALLET

From the demo you can show today to a real Google Wallet pilot in Gujarat

A click-by-click guide for the product, the setup, the customer journey, the venue sale and the launch work still ahead.

PREPARED 04 OCTOBER 2026 | VERSION 1.0

This guide describes the code that exists today and clearly marks the outside accounts, approvals and business decisions still needed. Estimates are planning ranges, not promises.

START HERE

Read this before you book a venue demo

Use the fictional-data buyer demo to sell the idea now. Use the real app only after you connect its database and provider accounts.

In this guide

Stage	What you will do
1-2	What is built, timing, and show the sales demo
3-4	Pitch, buyer questions, and record a pilot order
5-8	Accounts, Supabase, database, owner login
9-12	GitHub, Vercel, Google Wallet, public approval
13-18	End-to-end sign-off, venue launch, privacy, security, Apple later

The honest current state

- Built: an offline-friendly buyer demo using fake browser data; a Google-first Next.js app; separate owner/admin and staff views; adjustable venue colors, action labels, rewards and lifetime tiers; a member ledger; QR scanning; and a Google Loyalty Class/Object adapter.
- Added in this work: a public venue sign-up page at [/join/venue-slug](#), required consent, a starter privacy page, and per-venue sign-up rate limiting. The owner dashboard shows the shareable sign-up link.
- Checked: the production Next.js build completed successfully after these changes. This is a compile check. It is not an end-to-end test against a real Supabase database or Google Issuer.
- Not connected yet: no Supabase project or production database, no Vercel deployment, no Google Issuer or Google approval, and no Apple Wallet implementation. The privacy and program-terms pages are drafts.

How quickly?

You can present the fictional-data sales demo today. Allow roughly 1-3 focused working days to polish the meeting materials and pilot offer. A real test pilot is roughly 3-7 focused working days after the owner gives working account access, images and venue details. Google's approval to issue passes to every customer adds an outside wait that we cannot schedule. A broader, carefully operated launch is more like 4-8+ weeks of work, depending on legal review, support and account approval.

Can I take orders now?

Yes: collect venue interest and agree a conditional pilot, with the price and start date written down. Do not promise a live customer wallet, a fixed ROI, or a public Google pass until the app, business terms and issuer approval are ready. Do not take a deposit until the price, refund terms and delivery date are confirmed in writing.

1 / TODAY

Open the demo and walk a venue through it

The buyer demo runs on one laptop and uses invented sample members. It needs no Apple membership, Google Wallet account, Supabase keys, or install.

Start it on a Mac

1. Press **Command + Space**, type **Terminal**, and press Return.
2. Copy this command into Terminal and press Return: `python3 -m http.server 4173 --directory /Users/aaditya/Desktop/aikia.wallet/demo-service`.
3. Leave Terminal open. It is serving the local demo. Open Safari or Chrome and go to **http://localhost:4173**.
4. If macOS asks if Python may accept a connection, allow it for this private demo. Stop the server later by clicking Terminal and pressing **Control + C**.

Run the 5-minute story

1. Say: **AIKIA.WALLET gives a venue its own branded digital loyalty card. The venue chooses what earns progress, what the reward is, and which repeat customers unlock better benefits.**
2. Start on the **Owner** screen. Change the sample venue name and card color to match the buyer's business.
3. Open **Program**. Change the qualifying action, reward target, reward name, tier thresholds and benefits. Explain that a cafe can count coffees; a salon can count visits; a shop can count qualifying purchases.
4. Open **Members**. Show the sample member list, join experience and progress. Make clear that those names are invented and saved only in this browser.
5. Switch to **Staff**. Find a sample member by code or QR, add an action, then redeem a sample reward. Explain that authorized venue staff do this at the counter.
6. Close by asking what repeat customer behavior they want to encourage and proposing a short pilot after their program rules and price are agreed.

Never use real customer data in this demo

The demo uses local browser storage, not the cloud database. Its role switch is only for presenting the idea. Real names and sales figures belong in the connected live service after setup and venue approval.

For the code demo instead

Once the GitHub checkout and service credentials are ready, the live app is run from Terminal with `cd /Users/aaditya/Desktop/aikia.wallet/software` and `npm run dev`. Open **http://localhost:3000**. The demo and the connected application are two different products; the live app needs Supabase before sign-in or member records work.

2 / SELL CLEARLY

A simple pitch buyers can understand

Sell repeat visits and a branded member experience. Avoid technical words unless the owner asks.

30-second pitch

Say this

AIKIA.WALLET gives your regulars a digital loyalty card for your business. You choose the brand colors, what earns a stamp, the reward, and the higher membership levels. Your team can find a member, record a visit and redeem a reward from its staff screen. Customers can join from a link and, when the Google Wallet setup is approved, save their card to Google Wallet. We are starting with a small Gujarat pilot and will measure whether customers return more often.

Ask these five questions

- What do you want customers to do more often: visit, buy another item, book a repeat appointment, or try a premium service?
- How do you reward regulars today? Where does the current method break down?
- Roughly how many repeat customers and counter staff would try the pilot?
- What reward would feel valuable to the customer but still make financial sense for your business?
- Who can approve the pilot, the program rules and the monthly price?

Show the value with the venue's own numbers

Do not claim AIKIA will make a set amount of money. Work it out together. Use actual venue figures and write down the assumptions:

Extra visits x average spend = extra sales. Extra sales x gross margin = estimated extra gross profit. Estimated extra gross profit - reward cost - AIKIA fee - setup cost = estimated net benefit.

Example: if the owner says a repeat visit is worth Rs. 500 in sales and estimates a 40% gross margin, that visit contributes about Rs. 200 before the reward, software fee and other costs. These numbers are a made-up illustration. Use their data; measure the pilot; make no guarantee.

Handle common questions

Buyer asks	Clear answer
We already have paper cards.	Great. What is hard to track today? We can test whether a branded digital card makes repeat progress easier to see. Keep paper during the pilot if needed.
Does it take payments?	No. This product is for loyalty membership and rewards. It does not process card payments or replace your billing/POS system.
Will every phone work?	The first live pass is Google Wallet. Test the customer's phone before promising their experience. Apple Wallet is not built yet; staff can still look up a member code.
Will it increase revenue?	It is designed to encourage repeat behavior. We will agree a baseline and measure the pilot. Results depend on the venue, offer and customer response.
How much does it cost?	Use the approved written AIKIA price sheet. Do not invent a price at the meeting. Quote setup, monthly service and any pilot discount separately.

3 / START THE ORDER

Record a pilot request without overpromising

A signed conditional pilot sheet records buyer intent. It is not a substitute for a final contract, privacy terms or a payment receipt.

Before you leave the meeting

1. Ask for the legal business name, venue display name, one pilot location, owner/decision maker and a work email/phone number.
2. Write down the goal, current repeat-customer process, proposed qualifying action, sample reward, target start date and number of staff to train.
3. Show the owner and staff demo screens and record what they want changed: colors, logo, action wording, reward and tiers.
4. Agree the pilot scope, price, any setup charge, term, cancellation/refund policy and who supplies logo/legal copy. If the price sheet is not approved, label the request **INTEREST ONLY - PRICE TO BE CONFIRMED**.
5. Explain that public Google Wallet pass issuing depends on the issuer's business review. Until approval, a Wallet pass may say TEST ONLY and be limited to named test accounts.
6. Ask permission before taking or storing personal contact details. Send the venue a written summary and planned next step.

Pilot order / interest record

Field	Write in at the meeting
Venue + location	_____
Owner / decision maker	_____
Email / phone (with permission)	_____
Repeat action to reward	_____
Reward + tier ideas	_____
Pilot goal + baseline	_____
Number of staff / members	_____
Price / term / refund	_____
Target start / next meeting	_____
Status	Interest only / conditional pilot / contract signed / live

Buyer name / signature: _____ Date: _____

AIKIA representative: _____ Date: _____

4 / PRODUCT MAP

What the real software will do

The customer join flow, venue dashboard, staff workflow and wallet provider are connected by the venue's database.

Person	What they do
AIKIA owner	Creates the venue workspace; edits its look, qualifying action, reward, lifetime tiers and benefit text; invites the venue team; reviews its member list.
Venue manager / staff	Signs in through an email link; scans or enters a member's code; records one qualifying action; confirms and records a reward redemption.
Customer	Opens the venue's public join link; enters a name and agrees to the notice; sees a member code; saves the pass to Google Wallet if pass setup is working.
AIKIA server	Stores the member and ledger in Supabase, checks staff permission, then creates or updates the Google Wallet pass from current ledger values.

What the first live release includes

- Separate owner/admin navigation and a staff-only counter workflow.
- Venue-selected business type, brand color, action label, stamp/balance label, reward threshold and reward name.
- Lifetime tiers with custom names, minimum actions, accent colors and benefits. For the live Google Wallet card, color belongs to the venue's shared class; the demo can preview tier colors individually.
- Per-venue customer URL: /join/your-venue-slug. The member code can be typed; the Wallet pass barcode carries a random scan value.
- An append-only visit/reward ledger. The Google pass is a display of the data; it is not the authoritative record.

What is outside the first release

- Apple Wallet, payments, POS or billing integration, SMS/email marketing, subscription billing, customer self-service and automatic reward messages.
- A polished self-serve venue account signup and automated billing. For the first pilot, AIKIA manually sets up a venue and invites its owner.
- Automatic queue/retry for every failed Google sync, full monitoring and an independent security review. These are launch-hardening tasks, not nice-to-haves for scale.

India and payments

This product issues loyalty passes. It does not make Google Pay/UPI/card payments. Google's India help currently lists Wallet availability but says contactless payment is not available in India; a loyalty card in Wallet is a separate product. Confirm the customer's device and pass behavior during pilot testing.

5 / PREPARE ACCOUNTS

Collect these before the live setup

Create accounts in the AIKIA business owner's name. Keep the owner in control and grant developers only the access needed to set things up.

Need	Where it comes from	Who keeps it
GitHub repository access	github.com/SHXHMNS/aikia.wallet	AIKIA repo owner; reconnect the terminal GitHub login before upload.
Supabase project URL	Supabase Dashboard -> project -> Connect	Vercel setting; URL is not a secret by itself.
Supabase publishable key	Project Settings -> API Keys	Safe for browser use when prefixed NEXT_PUBLIC_.
Supabase secret key	Project Settings -> API Keys	Backend-only Vercel secret. Never add NEXT_PUBLIC_.
Google Wallet Issuer ID	Google Pay & Wallet Console	Vercel setting; numeric ID, not the private key.
Google service account JSON	Google Cloud -> IAM & Admin -> Service Accounts -> Keys	Vercel secret. Treat the private key like a password.
Program logo HTTPS URL	Test mark is included at /brand/aikia-wallet-program-logo.png; replace with approved venue artwork before launch	Public Vercel HTTPS image, no sign-in.
Join rate-limit secret	Generate in Terminal with openssl rand -hex 32	Vercel secret. Do not reuse a Google or Supabase key.
Business terms / privacy contact	AIKIA and each venue owner, with India legal advice	Must be final before collecting real members.

Please do not send passwords or private keys in chat

Use your password manager and provider secret settings. Do not paste the Supabase secret key or Google JSON private key into GitHub, an email, a screenshot, a support ticket or a public document. If a private key leaks, revoke it in Google Cloud and create a replacement.

Names of settings the app reads

NEXT_PUBLIC_APP_URL, NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, SUPABASE_SECRET_KEY, GOOGLE_WALLET_ISSUER_ID, GOOGLE_WALLET_SERVICE_ACCOUNT_JSON, GOOGLE_WALLET_PROGRAM_LOGO_URL, JOIN_RATE_LIMIT_SECRET, and WALLET_PROVIDER=google.

6 / DATABASE

Create the Supabase project

The database is where real venue, member, reward and staff data live. Create a development project first. Do not test with real customers.

1. Open supabase.com/dashboard and sign in to the AIKIA owner's account.
2. Choose the AIKIA organization from the organization picker. Click **New project**.
3. Name it **aikia-wallet-dev**. Generate a strong database password and save it in the password manager. You do not need to put that password in this app's environment variables.
4. For the region, choose **South Asia (Mumbai)** if it is offered for your account. Supabase says the primary region controls where the database is hosted; a region choice alone does not prove legal compliance.
5. Choose the available plan and click **Create new project**. Wait until the project has finished provisioning.
6. In the left menu, open **Project Settings** -> **API Keys**. Copy the Project URL and the **publishable** key into your password manager. Generate/copy the **secret** key for the server only.
7. Do not use the secret key in any page or file whose name starts NEXT_PUBLIC_. Supabase secret keys bypass normal row security checks; only the app server should hold one.

Run the database setup

1. In Finder, open the project folder Desktop/aikia.wallet/software/supabase/migrations.
2. In the Supabase Dashboard, click **SQL Editor** -> **New query**.
3. Open 0001_wallet_core.sql in a text editor. Select all, copy, paste it into the SQL Editor, and click **Run**.
4. Wait for the success message. Open another **New query**. Repeat with 0002_public_customer_join.sql.
5. If this Supabase project already has migration 0001, do not paste it again; run only migration 0002. If any SQL run fails, stop and keep the error. Do not manually delete production tables to retry.
6. Click **Table Editor**. Confirm you see venues, venue_team_members, venue_tiers, members, ledger_transactions, wallet_passes and public_join_limits.

Why two files?

0001 creates the venue, staff, tiers, member and ledger foundation. 0002 adds the public customer join fields and rate-limit table. SQL Editor is the beginner-friendly setup path. In later software releases, use versioned migrations through the Supabase CLI so the database and Git history stay in step.

7 / OWNER SIGN-IN

Turn on the secure email-link login

The owner and staff get a one-time sign-in link in email. A real venue should use a reliable email sender so links do not disappear into spam.

1. In the Supabase project, click **Authentication** -> **URL Configuration**.
2. Keep the allowed local address for development: `http://localhost:3000/**`.
3. After Vercel gives the project an address, set **Site URL** to that production address, for example `https://aikia-wallet.vercel.app`.
4. Under **Redirect URLs**, add `https://aikia-wallet.vercel.app/**`. Add the exact Vercel team/account preview pattern only if you will use preview deployments. Save the changes.
5. Open **Authentication** -> **Sign In / Providers** -> **Email**. Keep email sign-in enabled. For a public launch, configure and verify a custom SMTP sender and check the email template and sending limits.
6. Use the live app's **Sign in** link, enter the owner's work email, open the Supabase email and click its secure link. The account can then create the first venue workspace.

Do not create customers as staff

Only venue owners/managers get operator accounts. Customers join from the public venue URL. Invite staff using their own named work email so an owner can remove one person's access without changing everyone else's password.

Local-only setup

Copy `software/.env.example` to `software/.env.local`. Open `.env.local` in TextEdit or another editor and fill the Supabase values. Save it. The repository ignores `.env.local`; still check `git status` before every push. Use `npm run dev` from `software`. Do not upload the file as a GitHub document.

8 / PUBLISH THE APP

The code is on GitHub; connect Vercel next

The public repository now contains the application and this guide. Vercel will build the software folder and make it available as a website.

GitHub checkpoint: complete

The local checkout is connected to origin/main. Commit 9f75396 (Add customer enrollment and launch runbook) has been pushed to github.com/SHXHMNS/aikia.wallet. The PDF and software files are there. Do not create a second repository or add another origin.

How you publish the next code change

1. Open **Terminal** and run `cd /Users/aaditya/Desktop/aikia.wallet`.
2. Run `git status --short`. Check the list. The repository ignores `.env.local`; never force-add it, a private key or customer data.
3. Run `git add -A` to stage the changes, then `git status --short` again to review exactly what will be saved.
4. Run `git commit -m "describe your change"` to make a saved version.
5. Run `git push origin main` to upload that version. The computer may ask you to complete GitHub sign-in.
6. Open github.com/SHXHMNS/aikia.wallet in a browser and confirm the new commit appears on **main**.

Connect Vercel to the software folder

1. Open vercel.com/dashboard and sign in. Click **Add New** -> **Project**.
2. Under Import Git Repository, find **SHXHMNS/aikia.wallet**. If it is absent, choose **Adjust GitHub App Permissions** and allow Vercel to see that repository.
3. Click **Import**. On Configure Project, open **Root Directory** -> **Edit** and choose software. Framework should show Next.js. Leave the build command at its detected default.
4. Before deploying, add the environment values on the project. Start with Supabase and rate-limit values. Keep Google pass values empty until its account is ready.
5. Click **Deploy**. Wait for the green ready state, then open the assigned `vercel.app` link and `/api/health` in a new tab.

9 / DEPLOYMENT SETTINGS

Put the settings in Vercel

Set each value for the right environment. Do not point a test preview at real venue data or expose a secret with the NEXT_PUBLIC_ prefix.

Name	Value / rule	Public?
NEXT_PUBLIC_APP_URL	Exact deployed HTTPS site origin, with no trailing page path	Public
NEXT_PUBLIC_SUPABASE_URL	Supabase Project URL	Public
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Supabase publishable key	Public
SUPABASE_SECRET_KEY	Supabase secret key (server only)	SECRET
JOIN_RATE_LIMIT_SECRET	Fresh random output from openssl rand -hex 32	SECRET
WALLET_PROVIDER	google	Public setting
GOOGLE_WALLET_ISSUER_ID	Issuer ID from Google Pay & Wallet Console	Not a key
GOOGLE_WALLET_SERVICE_ACCOUNT_JSON	Complete service account JSON key	SECRET
GOOGLE_WALLET_PROGRAM_LOGO_URL	Public HTTPS logo URL that loads without login	Public

For every row: in Vercel open the project -> **Settings** -> **Environment Variables** -> **Add New**. Enter the exact name and value, select the environment, and click **Save**. Mark Supabase secret, join-rate secret and Google service JSON as sensitive when available. Add separate values for Preview and Production; do not connect test pages to real member data.

- Set Production NEXT_PUBLIC_APP_URL to the exact HTTPS production host Vercel assigned, such as `https://aikia-wallet.vercel.app`. Do not include /dashboard or a trailing route. Leave Google secrets out of Preview until you have configured a stable Preview hostname and its matching origin.
- To make the join-rate secret, open a second Terminal window, run `openssl rand -hex 32` and paste its output into Vercel. Keep it private and do not reuse it for another system.
- Paste the complete service-account JSON into the Vercel value field. It may contain line breaks; preserve the full JSON. Never put it in a GitHub issue or source file.
- After all settings are saved, open **Deployments**, click the latest deployment's three-dot menu and choose **Redeploy**. Environment changes do not change a deployment that was already built.
- After a custom domain is added, update NEXT_PUBLIC_APP_URL and Supabase's Site URL/Redirect URLs to the final domain, then redeploy again.

Quick read-only check

Open `https://YOUR-DOMAIN/api/health`. It reports true/false setup indicators for the database, Google Wallet and join rate limit. It must never display the secret key or service-account JSON. A true value only means the settings are present; make a real pass and staff test before launch.

10 / GOOGLE WALLET

Create the Issuer and service account

Google Wallet issues a venue Loyalty Class and an individual Loyalty Object per member. The app signs a save link on the server.

Create the issuer

1. Open [Google Pay & Wallet Console](#) using the business owner's Google account.
2. Choose the Google Wallet API path and start issuer onboarding. Enter the true legal/business identity and public-facing name, plus the requested contact and business profile information. Save the numeric **Issuer ID** in the password manager.
3. Complete the Business Profile and its website, terms and privacy URLs. Do not use the draft AIKIA privacy/terms pages as final customer terms. Have approved pages on the real domain first.
4. Open [Google Wallet API in Google Cloud](#) and select the Cloud project used for the issuer. Click the project name at the top if needed, search the API Library for **Google Wallet API**, open it and click **Enable**.
5. In Google Cloud, open **IAM & Admin** -> **Service Accounts** -> **Create Service Account**. Name it `aikia-wallet-server`, continue through the prompts and finish.
6. Open the new service account -> **Keys** -> **Add Key** -> **Create new key** -> **JSON** -> **Create**. Store the downloaded JSON securely. Do not open it in a screen-share or upload it to GitHub.
7. Copy its `client_email`. Return to Google Pay & Wallet Console, open the Wallet API account's user/access settings, add that email as a **Developer**, and save. Wait for Google's access to take effect.
8. In Vercel add the Issuer ID and full JSON to the matching settings from the prior page. For an initial test, set the logo to `https://YOUR-PRODUCTION-HOST/brand/aikia-wallet-program-logo.png`. Save and redeploy. This is a simple AIKIA placeholder, not the final venue logo.

Who owns the issuer?

For this first controlled pilot, one AIKIA-operated issuer creates a separate branded loyalty class for each venue. That is the current code path. Before signing many venues, confirm in writing that this issuer model fits Google's current policies and the venue contracts. A class color is shared for that venue; different member tiers are shown as benefits and status, not as separately colored live classes.

11 / TEST AND ASK TO GO PUBLIC

First create a test pass, then request publishing access

A brand-new issuer starts in Demo Mode. Google limits who can save those passes until it reviews and approves the issuer.

Add test people

1. In the Google Wallet console, open the issuer's **Users** or test-account section and add the owner's Google account as an Admin/Developer or test user. Add one Android tester's Google account too.
2. Wait until they can sign in to Google Wallet. A personal Gmail or other Google identity used on the test phone must match the identity you authorized.
3. In the AIKIA app, sign in with the venue owner email, create the pilot venue using the real display name and approved slug, then open **Program** and set its logo URL, brand color, action, reward and tiers. Save the settings to create the pass class.
4. Open **Members** and add a test member, or open the public `/join/your-slug` link on the test phone. Tap **Add to Google Wallet**, choose the approved tester account and save the card.
5. At the venue dashboard, open **Staff**. Scan the pass QR using an HTTPS page on a camera-capable device, or manually enter its member code. Record a test action. Re-open the pass and confirm its displayed balance/tier updates.
6. Record a reward, redeem it once with staff, retry the same action once to make sure it does not give another stamp, and confirm the ledger and pass agree.

Request public access

1. Make sure the Issuer Business Profile is complete, the real business website and privacy/terms pages work, and at least one Loyalty Class has been created.
2. Open Google Pay & Wallet Console -> **Google Wallet API**. Find the **Get publishing access** panel and click **Request publishing access**.
3. Answer the questions with the real AIKIA product and venue use case. Submit the form and wait for Google to review it. Do not promise the buyer an approval date.
4. After approval, test again from an ordinary Google account that is not an issuer admin or test account. Confirm the pass no longer carries a TEST ONLY label.

India phone check

The official Google Wallet help page lists Google Wallet availability for India but says contactless payments are not supported. These are loyalty passes, not tap-to-pay cards. Test the exact Android devices, Google accounts, pass language and in-store scanning process your pilot customers use.

12 / PUT YOUR BRAND ONLINE

Set the final address, logo and venue program

A default Vercel URL is fine for the first internal test. A real venue pilot should use a stable public URL and artwork the venue owns or is allowed to use.

1. In Vercel, open **Settings** -> **Domains**. Add the AIKIA domain you control and follow Vercel's displayed DNS instructions at your domain registrar. Wait for Vercel to mark it valid and issue HTTPS.
2. Update Vercel's NEXT_PUBLIC_APP_URL to the final HTTPS site. Save, then redeploy.
3. In Supabase Authentication -> URL Configuration, change Site URL to the final address. Keep localhost for development and add the final site's /** redirect pattern. Save.
4. The app includes a simple AIKIA placeholder at software/public/brand/aikia-wallet-program-logo.png. It will load at https://YOUR-DOMAIN/brand/aikia-wallet-program-logo.png. Open that address in a signed-out browser window. Before a venue launch, replace this test mark with artwork the venue approves, redeploy, and set the exact new URL in Vercel as GOOGLE_WALLET_PROGRAM_LOGO_URL.
5. For each pilot venue, get its logo permission, display name, primary color, program terms, reward details and manager emails in writing. Use the owner dashboard to edit branding, action, reward and tiers.
6. Before real customers join, create a second Supabase project named aikia-wallet-production and run migrations 0001 and 0002 there. In Vercel, set Production variables to the new project's URL, publishable key and secret key. Keep Preview pointed at the dev project and keep Google secrets out of Preview unless its own hostname is configured.
7. Give the venue its own URL path such as https://wallet.example.com/join/cafe-noor. Open that link on a phone. In **Members**, copy the customer sign-up link to share. To make a counter QR, turn that public URL into a QR code using an approved design/print workflow, then test scan it yourself before printing.

Keep the promise narrow

The owner can change their theme and reward rules. The live Google Wallet class uses one venue color. The demo shows tier color variations as a presentation preview, but the live adapter does not issue a separate Wallet class for every tier.

13 / REAL CUSTOMER JOURNEY

Click through this acceptance checklist

Do these with fictional data in a Preview deployment and authorized Google Wallet testers first. Record each pass/fail and fix failures before a venue sees it.

#	Open / do	What must happen
1	Open /api/health on the site.	Database and join-rate settings show configured. Google is configured after its settings are present.
2	Open /login; enter the owner email; open the received link.	The owner reaches the dashboard. A logged-out visitor cannot open owner or staff API routes.
3	Create the test venue, adjust color/program, set tiers; refresh.	The saved settings remain. The Wallet class is created or updated.
4	Copy the public link from Members; open it on a test phone; sign up with a fake name and accept consent.	One member is created. The customer sees their code and an Add to Google Wallet link when Wallet is ready.
5	Try signing up repeatedly from the same network in 10 minutes.	The server blocks excess attempts. Confirm the customer sees a calm retry instruction.
6	Open the staff account on another device. Scan the Wallet barcode or enter the test code.	Staff can find only this venue's member and no cross-venue member.
7	Record one purchase, then repeat a network request with the same idempotency key.	The member receives only one qualifying action. The server ledger is the source of truth.
8	Redeem one earned reward and retry the same redemption.	Only one reward is removed. Staff see a clear success/error state.
9	Change the venue action/brand/tier; inspect the pass again.	Correct theme and reward display. No other venue's pass or member changes.
10	Open the site in a second browser/account and test expired invite, revoked staff and wrong-venue links.	No unauthorized action succeeds. Ask a security reviewer to check this before scale.

A build is not a pass test

The successful code build checked TypeScript and made the Next.js pages. It did not contact Supabase or Google because AIKIA does not yet have the owner's project keys and Issuer. The checklist above can only be marked complete after a real connected test.

14 / ONBOARD THE FIRST VENUE

Set up one cafe before selling ten

A small pilot teaches you where the app and service process need improvement. Keep the first offer simple and write down what happens each week.

1. Get a signed pilot scope, approved price, venue brand image permission and venue-specific reward rules.
2. Create the venue as AIKIA owner. Use a slug that is short, lowercase and easy to say. For example, cafe - noor .
3. Open Program and set the venue's legal/display names as requested, primary color, logo, qualifying action, balance label, reward count and reward text.
4. Set lifetime tiers in a ladder the owner can explain. Put each tier threshold above the last and state the exact benefit and its exclusions.
5. Open Team and invite the manager's named work email. The manager checks their inbox, signs in and tests the staff view. Never share the owner's login.
6. Open Members and copy the join link. Test it on the public site, then make one sign or QR print. Verify the QR with a phone camera. Put a second copy in the staff playbook.
7. At opening, have staff sign in on the device they will actually use. Practice one fake signup, one qualifying action and one reward redemption.
8. At close, compare the AIKIA ledger with the venue's own sales/ticket count for the test day. Investigate mistakes while they are still easy to remember.

Measure these weekly

Metric	What it tells the owner
Members who joined	Is the counter invite working?
Members with a second visit	Are customers coming back after joining?
Qualifying visits/actions per member	Is the reward ladder attainable and understood?
Rewards earned vs. redeemed	Does the reward cost match the benefit?
Staff actions with a correction/support request	Is the workflow clear and are mistakes being caught?

Ask the owner to compare members' repeat visits with a similar pre-pilot period or a similar non-member group when possible. A short pilot may be too small to prove cause. Never call a correlation proof that the wallet caused the sale.

15 / INDIA PRIVACY AND CUSTOMER PROMISES

Replace the starter legal copy before live enrolment

The current /privacy and /terms pages say they are starter copy. That warning must not be removed until the real owner, terms and customer contact are inserted and approved.

Before the first real member joins

- Decide which legal company operates AIKIA and which venue decides the purpose and rules of each loyalty program. Get India legal advice on the roles and written data-processing agreement.
- Publish accurate venue/AIKIA business identity, customer contact details, what data is collected (name, member code, activity, progress, tier, consent time, and the short-lived keyed network-address fingerprint used to limit sign-up abuse), why, who receives it, how long it is kept, and the way a customer can ask for a correction or removal.
- State clear reward rules: what counts as a visit, excluded items, redemption steps, expiration date if any, value and limits, whether offers combine, how disputes work and what happens if the program ends.
- Make the join checkbox optional until the customer chooses to join. Do not pre-check consent. Keep the chosen notice version/time with the membership.
- Make a written support process for data requests, deletion, incorrect stamps, a stolen phone, a suspected breach and removing staff access. Do not promise a legal response time you have not checked.
- Ask counsel to review the Digital Personal Data Protection Act and current rules, the Google Wallet issuer terms, venue contracts, age/guardian policy, taxes, consumer protection and reward expiry rules. This guide is not legal or tax advice.

The starter notice currently says the venue runs the program and that AIKIA supplies software. A lawyer must check that this is accurate for your business. The privacy route only contains draft text; it does not provide a deletion button or a verified legal contact form. The account-removal procedure must be made real before customer launch.

Official starting point: [MeitY's Digital Personal Data Protection Rules 2025 page](#). Check the final Act, notified rules and start dates with a qualified India privacy lawyer before launch.

Minimize data

The first join page asks only for a name and consent. Do not start collecting phone numbers, birthday, location or payment data just because a buyer asks. First write down why the data is needed, how you protect it and how to delete it.

16 / OPERATE IT SAFELY

Before moving from pilot to public launch

The software has important controls, but no code can promise zero breaches. Reduce the data collected, limit access, keep backups and rehearse failures.

Area	Required pilot habit
Access	Give each owner and worker a separate account. Remove a departed worker promptly. Review owners/admins monthly.
Secrets	Use Vercel's server-side environment settings. Rotate keys if they leak. Never paste a secret into GitHub or the browser.
Customer data	Use fake data in demos; real names only after approved privacy text and venue agreement. Restrict exports and avoid spreadsheets of member history.
Abuse	The current sign-up has a small server-side per-IP limit. Add managed bot protection/WAF limits before public marketing. Monitor signups and unexpected use.
Pass sync	The ledger remains authoritative. A Google API timeout may leave a pass stale. Provide a manual staff fallback and a daily check until queued automatic retries exist.
Backups	Confirm the Supabase plan's backups and restore choices, access to those backups, and the price. Do a restore rehearsal before storing valuable member history.
Monitoring	Add monitoring for failed login, signup spikes, provider errors, database capacity, and site outages. The health page only reports configuration; it is not full monitoring.
Incident plan	Write who can disable a join link, revoke a Google key, remove a staff login, contact affected venues, preserve facts, and get legal help.
Release	Have a developer review code, dependencies, database RLS and provider permissions. Use a qualified independent security reviewer before a broad launch.

Known pilot limits

The app does not yet offer billing, subscription plans, POS integrations, a retry queue, customer self-service, Apple passes or fully automated venue provisioning. Staff record a visit manually. Describe these limits honestly when taking a conditional pilot order.

17 / APPLE LATER

Add Apple Wallet after the Google pilot

The code has a provider boundary, but the Apple adapter is only a placeholder. Do not quote Apple support as an existing feature.

1. Enroll the business in the Apple Developer Program using the legal business identity and an Account Holder who can sign Apple's agreements.
2. In Certificates, Identifiers & Profiles, create a Pass Type ID identifier for AIKIA Wallet passes.
3. Create the Pass Type ID certificate, download the certificate and private key, and store them securely. Record the Team ID and certificate password if one is set.
4. Ask a developer to implement and test Apple's signed .pkpass package, barcode and pass update service, certificate rotation, device installation, and Apple push updates behind the existing provider interface.
5. Add Apple secrets as server-only Vercel variables. Never commit the certificate private key or password. Add a fresh iPhone pilot path and verify scan, reward update, revoke and renewal flows.
6. Update the buyer pitch, privacy page, support instructions, Vercel settings and PDF only after the Apple adapter passes a real device test.

Apple enrollment alone does not finish the integration

Buying the Developer Program is an account prerequisite. The pass creation, certificate handling and update code still need to be written and tested. Keep the current Google-only pilot narrow while this is built.

Apple's official pass setup guide: [Create Wallet identifiers and certificates](#). Pass design guide: [Building a pass](#).

18 / YOUR NEXT WEEKS

A realistic route from demo to first paying venue

The fastest path is to keep the first pilot small, get the provider accounts started early, and avoid waiting for Apple before the Google test works.

Phase	Work	Planning time
A. Sales demo	Use fictional demo now; confirm 3-5 interested Gujarat venues; ask what reward they want; agree written pilot price and scope.	Today to 1-3 workdays
B. Accounts	Owner provides GitHub access, Supabase dev project, Vercel project, approved brand image, privacy/terms draft and Google Wallet issuer access.	1-3 workdays, depending on access
C. Connected pilot	Push repo; apply SQL; deploy; set email URLs; connect issuer; invite venue and Android test accounts; run every acceptance check.	About 3-7 focused workdays after access
D. Public Google pass	Complete issuer profile, class and test pass; request publishing access; wait for Google's review; test with ordinary customers after approval.	External review wait varies; no date promised
E. First venue	Sign final service/privacy terms, tune reward ladder, train staff, publish QR/join link, support and measure one venue.	Start as soon as legal and public pass gates clear
F. Broader launch	Add retry, backups/restore drill, alerts, bot protection, security review, venue billing and Apple support as separate releases.	Roughly 4-8+ weeks of engineering and operations

These are planning estimates, not an employment contract or launch promise. The 3-7 day estimate assumes the owner replies promptly, creates working provider accounts and gives the developer correct access. Google's approval queue and business verification are outside the coding estimate.

What you can do next, in order

- Tomorrow: show the sample demo to five venue owners. Gather permissioned interest and the repeat-action/reward they would actually buy.
- Pick one venue and agree a realistic pilot price, scope and start conditions. Ask that owner for their correct business display name and logo.
- Fix GitHub terminal access and push the app branch; only then create the Vercel project from the software directory.
- Create Supabase dev, apply both SQL files, set email redirect URLs and check a test owner login.
- Start Google Wallet issuer onboarding now because publishing approval can take extra time. Add a tester and create the first test Loyalty Class/Object.
- Run the acceptance checks, finish legal pages with counsel and publish the public pilot only after venue terms, privacy contact, Google approval and staff training are all ready.

REFERENCE

Fix the common setup snags

When a step fails, capture the exact non-secret error text and the screen name. Never capture a password or private key.

What you see	What to check first
Live home says schema missing	In Supabase, confirm 0001 and 0002 both ran successfully in the correct project. Reload the site.
Owner login link errors	Authentication -> URL Configuration has the exact Vercel host in allowed Redirect URLs. Confirm Supabase email sender limits and spam folder.
Vercel cannot see repo	GitHub app installation for Vercel is granted access to SHXHMNS/aikia.wallet; code is pushed to main; Root Directory is software.
Join page says temporarily unavailable	Vercel has SUPABASE_SECRET_KEY and JOIN_RATE_LIMIT_SECRET; 0002 SQL ran; latest deployment was redeployed after changing settings.
No Add to Google Wallet link	Issuer ID, service account JSON, HTTPS logo URL and NEXT_PUBLIC_APP_URL are present. Check Vercel function logs for a provider error without copying secrets.
Google rejects save link	Service account email has Issuer Developer access; API is enabled; test user's Google account is authorized in Demo Mode; class was created.
Pass says TEST ONLY	Issuer is still in Demo Mode. Finish Business Profile, create a class, request publishing access, and wait for Google approval.
Staff camera cannot scan	Use HTTPS (or localhost), grant browser camera permission, use a supported browser, or type the member code. Keep the code as fallback.

Official setup pages

- [Google Wallet issuer onboarding](https://developers.google.com/wallet/retail/loyalty-cards/getting-started/issuer-onboarding) - <https://developers.google.com/wallet/retail/loyalty-cards/getting-started/issuer-onboarding>
- [Google Wallet REST API authentication](https://developers.google.com/wallet/retail/loyalty-cards/getting-started/auth/rest) - <https://developers.google.com/wallet/retail/loyalty-cards/getting-started/auth/rest>
- [Google Wallet publishing access](https://developers.google.com/wallet/retail/loyalty-cards/test-and-go-live/request-publishing-access) - <https://developers.google.com/wallet/retail/loyalty-cards/test-and-go-live/request-publishing-access>
- [Google Wallet launch checklist](https://developers.google.com/wallet/retail/loyalty-cards/test-and-go-live/launch-checklist) - <https://developers.google.com/wallet/retail/loyalty-cards/test-and-go-live/launch-checklist>
- [Google Wallet availability in India](https://support.google.com/wallet/answer/12060037?co=ADWORDS.BillingCountryCode%3DIN&hl=en-au) - <https://support.google.com/wallet/answer/12060037?co=ADWORDS.BillingCountryCode%3DIN&hl=en-au>
- [Supabase Next.js quickstart](https://supabase.com/docs/guides/getting-started/quickstarts/nextjs) - <https://supabase.com/docs/guides/getting-started/quickstarts/nextjs>
- [Supabase API key types](https://supabase.com/docs/guides/getting-started/api-keys) - <https://supabase.com/docs/guides/getting-started/api-keys>
- [Supabase Auth redirect URLs](https://supabase.com/docs/guides/auth/redirect-urls) - <https://supabase.com/docs/guides/auth/redirect-urls>
- [Supabase available regions](https://supabase.com/docs/guides/platform/regions) - <https://supabase.com/docs/guides/platform/regions>
- [Vercel monorepo root directory](https://vercel.com/docs/monorepos) - <https://vercel.com/docs/monorepos>
- [Vercel Git deployments](https://vercel.com/docs/git) - <https://vercel.com/docs/git>
- [Vercel environment variables](https://vercel.com/docs/environment-variables/managing-environment-variables) - <https://vercel.com/docs/environment-variables/managing-environment-variables>
- [Vercel request headers and client IP](https://vercel.com/docs/headers/request-headers) - <https://vercel.com/docs/headers/request-headers>
- [MeitY DPDP Rules 2025](https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-) - <https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025->
- [Apple Wallet identifiers and certificates](https://developer.apple.com/help/account/capabilities/create-wallet-identifiers-and-certificates/) - <https://developer.apple.com/help/account/capabilities/create-wallet-identifiers-and-certificates/>

Project folder: /Users/aaditya/Desktop/aikia.wallet | Software app: software/ | Database migrations: software/supabase/migrations/